

TB vs UVM map

You have written flat directed testbenches, clock and reset, tasks, fork-join, classes, constraints, cover

Six pairs to connect

- The lab lines up six classic roles with UVM counterparts
- Pin drive maps to driver plus virtual interface
- Fixed stimulus lists map to sequence plus sequencer
- Wire peeks with display map to monitor plus analysis port
- Inline expect maps to scoreboard
- One flat testbench module maps to environment plus agent hierarchy

Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / Basic TB vs UVM map

INTERACTIVE TOOL

Basic TB vs UVM map

Map **wiggle-the-pins** testbenches onto UVM roles: sequences, agents, drivers, monitors, and scoreboards. Starter: **UART TX byte** — pin drive already linked to the driver.

Starter example: UART TX byte with *pin wiggle* ↔ *driver* selected and marked mapped. Click other pairs, Mark mapped, or Demo all.

Load starter example

Challenges 0 / 22 cleared

Quiz: goal: This lab's main idea is...

☐ map classic pin TB roles onto UVM components

☐ compile Accellera UVM in the browser

☐ synthesize a UART VIP

☐ replace all \$display with SVA

Show hint

Check

Next

Idle

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

Core ideas

Pins → transactions

UVM moves intent from edges to items.

Split roles

Drive, watch, and check are separate components.

Agent

Driver + monitor + sequencer around one interface.

Reuse

Factory and VIP beat copy-paste TBs.

Browser lab starter

learn_sv_tb — tb vs uvm map

Real shell practice

- In the real shell track, open this module's examples prompts
- What you do today in directed style, and which UVM role would absorb that job
- Optional stretch
- No UVM compile required, the exercise is naming and boundaries

Pitfalls to watch

- Do not assume UVM replaces thinking, it replaces ad-hoc structure with reusable components
- Do not map buzzwords without behavior
- Do not skip directed TB fluency
- And do not treat this sketch as a production testbench generator, it is a concept map
- Real UVM still needs phases, objections, config database, and offline sim discipline

Your turn

- Complete the checklist for at least one track
- In the browser
- Take one habit from your own directed TB and write the UVM role that would own it
- Take the short quiz